RUNBOOK — Assessment 3, start to finish

The order to work in, and how to check each stage before moving on.

This runbook assumes you have written the files `TASKS.md` says are yours. It tells you when to deploy each piece and how to confirm it worked; it does not tell you what to put in them. The skeletons do that, each one carrying the requirement, the traps and the check for its own file.

Read in this order:

1. `TASKS.md` — what is given and what you must write
2. `docs/CLOUD-CONTRACT.md` — what stage 1 has to produce, on any cloud
3. this file — the order of operations
4. `docs/TROUBLESHOOTING.md` — when something does not work
5. `docs/CRITICAL-EVALUATION-TEMPLATE.md` — when you come to write Task 8

Placeholders used throughout:

Placeholder	Meaning	Example
`<user>`	your username, lowercase	`jsmith`
`<cloud>`	`aws`, `azure` or `catalyst`	`azure`
`<region>`	your permitted region	`australiaeast`
`<host>`	the hostname stage 1 outputs	`jsmith-shop.australiaeast.cloudapp.azure.com`
`<admin>`	the VM login account	`ubuntu`, or `azureuser` on Azure

0. Choose your cloud and check your quota

You may use **AWS**, **Azure** or **Catalyst Cloud**. All three are equally creditable. Only stage 1 differs; everything above it is identical.

Read `docs/CLOUD-CONTRACT.md` first — it states what stage 1 must produce.

Five minutes checking quota here saves an afternoon.

AWS

```
aws sts get-caller-identity          # credentials current? they expire hourly
aws ec2 describe-account-attributes --attribute-names supported-platforms
```

Learner Lab permits `us-east-1` and `us-west-2` only, and volumes up to 100 GiB.

Azure

```
az login
az account show --output table

az account list-locations --query "[].name" -o tsv | sort      # allowed regions
az vm list-usage --location <region> -o table | grep -i "Total Regional"
az vm list-skus --location <region> --size Standard_B2 \
  --query "[].{name:name, restrictions:restrictions[0].reasonCode}" -o table
```

A blank `restrictions` column means usable. Can you create a resource group?

```
az group create --name <rg>-probe --location <region> && \
  az group delete --name <rg>-probe --yes --no-wait
```

If that fails, set `create_resource_group=false` and use the group you were given.

Catalyst Cloud

```
source ~/openrc.sh                    # downloaded from the dashboard
openstack token issue                 # credentials work?

openstack quota show                  # quotas are PER PROJECT
openstack flavor list                 # confirm c1.c2r8 exists
openstack image list --public | grep -i ubuntu
openstack network list --external     # confirm the public-net pool name
```

Whichever you chose

You need **2 vCPU and 8 GiB**: `t3.large` (AWS), `Standard_B2ms` (Azure), `c1.c2r8` (Catalyst). A 4 GiB VM will evict Pods once the observability stack starts, and it looks like an application bug.

From here on, `<cloud>` means `aws`, `azure` or `catalyst`.

---

1. Set up your repository

```
git init assessment3 && cd assessment3     # or clone the private repo you were
given
# unpack the starter here
./scripts/check-progress.sh <cloud>
git add -A && git commit -m "starter repository"
```

Keep this in **private** source control from the first commit, and commit as you go. Criterion 1 assesses your workflow; a single "final" commit is poor evidence of one, and a marker can tell the difference.

Delete the two `terraform/infra-*` directories for the clouds you did not choose. Decide now — moving later means rewriting stage 1.

---

2. Stage 1 — build the infrastructure

Write first: `terraform/infra-<cloud>/main.tf`, then fill in the four values in `outputs.tf`. The skeletons in both files list the requirements.

```
cd terraform/infra-<cloud>
ssh-keygen -t rsa -b 4096 -f k8s-key -N ''     # this deployment only

terraform init
terraform fmt -check
terraform validate
terraform plan -var="username=<user>" -var="ssh_source_cidr=$(curl -s
ifconfig.me)/32"
```

Read the plan before you apply it. Check the firewall rules are the three you intend. It is much cheaper to find a mistake here than after the apply.

```
terraform apply   # same -var flags
terraform output
terraform output -raw shop_host     # note this; you need it repeatedly
```

If `terraform output` prints anything containing `TODO`, `outputs.tf` is not finished and nothing downstream will work.

Evidence for Task 1: the plan, the apply, and `terraform output`.

---

3. Wait for the VM to finish bootstrapping

Terraform returns when the provider says the VM exists. The supplied `cloud-init.yaml` is still working for several minutes after that.

```
ssh -i k8s-key <admin>@<host>
sudo tail -f /var/log/cloud-init-output.log
ls -l /var/lib/cloud/bootstrap-complete      # the marker it writes when done
```

Then confirm the cluster **and the storage**:

```
microk8s status --wait-ready
kubectl get nodes
lsblk
```

```
df -h /datadisk
findmnt /datadisk
```

`/datadisk` must be your dedicated data disk. On Azure in particular, do not confuse it with the temporary disk the platform mounts at `/mnt` — that one is destroyed on deallocation, and a PersistentVolume placed there disappears without warning.

Evidence for Task 1: `lsblk` and `df -h /datadisk`.

---

4. Copy the project to the VM

```
rsync -av --exclude '.terraform' --exclude '*.tfstate*' --exclude 'k8s-key*' \
  ./ <admin>@<host>:~/assessment3/
```

Or clone your private repository on the VM.

---

5. Stage 2 — install the platform

Write first: `terraform/platform/main.tf`.

Run this **on the VM**, not your laptop. That is deliberate, and the skeleton explains why.

```
ssh -i k8s-key <admin>@<host>
cd ~/assessment3/terraform/platform
terraform init
terraform apply -var="grafana_admin_password=<choose-something>"
```

Allow five to ten minutes; it pulls a lot of images.

```
helm list -A              # three releases, at the pinned versions
kubectl get pods -A       # all Running or Completed
```

Evidence for Task 2: the apply, `helm list -A`, all platform namespaces healthy.

---

6. Build the images and deploy the application

Write first: the two Dockerfiles, and `manifests/01`, `02`, `04`, `05`.

```
cd ~/assessment3
chmod +x scripts/*.sh
./scripts/build-images.sh v1
curl -s http://localhost:32000/v2/_catalog        # all three images present?
```

```
kubectl apply -f manifests/00-namespace.yaml
kubectl apply -f manifests/01-config.yaml
kubectl apply -f manifests/02-redis.yaml
kubectl apply -f manifests/03-catalog.yaml
kubectl apply -f manifests/04-orders.yaml
kubectl apply -f manifests/05-frontend.yaml

kubectl -n shop get all,pvc,configmap,secret
kubectl -n shop rollout status deploy/frontend
```

Apply them one at a time the first time through. `kubectl apply -f manifests/` on the whole directory is quicker once everything works, but it hides which file caused an error.

Evidence for Tasks 3 and 4: the builds, the registry catalogue, and `kubectl -n shop get all,pvc,configmap,secret`.

---

7. Publish it over HTTPS

Read first. Neither the Ingress resource nor cert-manager was covered in the labs, so start here rather than with the YAML:

```
# 1. the concepts
less docs/INGRESS-AND-TLS-PRIMER.md

# 2. a working Ingress, plain HTTP, no TLS. Apply it, watch it route,
#    break it the four ways the file suggests, then delete it.
kubectl apply -f docs/examples/EXAMPLE-ingress.yaml
kubectl -n shop get ingress
curl -s http://<your-public-ip>/products | head
kubectl delete -f docs/examples/EXAMPLE-ingress.yaml
```

Delete it before you go further. It publishes a ClusterIP service over plain HTTP, which is the opposite of what you are building.

Write first: `manifests/06-tls.yaml`, then `07-ingress.yaml`. Expect this to take longer than the previous two steps together.

```
./scripts/render.sh <host>            # substitutes __SHOP_HOST__
kubectl apply -f manifests/06-tls.yaml
# wait for your CA Certificate to become Ready BEFORE applying the leaf
kubectl apply -f manifests/07-ingress.yaml
kubectl -n shop get certificate -w
```

Then check it from outside:

```
curl -sk -o /dev/null -w '%{http_code}\n' https://<host>/
curl -s  -o /dev/null -w '%{http_code} -> %{redirect_url}\n' http://<host>/
```

Expect `200`, then a redirect to the `https://` URL.

Open `https://<host>/` in a browser. **It will warn.** That is the correct outcome — the certificate is signed by a CA no browser has heard of. Prove the chain is sound instead of clicking through:

```
echo | openssl s_client -connect <host>:443 -servername <host> 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Subject and issuer must **differ**. If they are the same you built a single self-signed certificate rather than a chain, and Criterion 5 caps at a bare pass however well the site works.

Then export your CA from its Secret and validate against it with `curl --cacert`. A `200` without `-k` is the evidence that the chain is sound and only the trust anchor is missing. Put that command in your report.

Evidence for Task 5: `describe certificate`, the `openssl` output, the redirect, the `--cacert` validation, and a browser screenshot.

---

8. Run the self-check

```
./scripts/verify.sh <host>
```

Every `FAIL` is a mark you are about to lose. Read this script early, not late — it names the objects your manifests are expected to create, so it is a specification as much as a test.

---

9. Check the monitoring

Write first: `monitoring/alerts.yaml`, and add a business metric of your own to one of the services.

```
kubectl apply -f monitoring/servicemonitors.yaml
kubectl apply -f monitoring/alerts.yaml
```

Prometheus and Grafana are not exposed publicly — by design, which is why your firewall has no rule for them. Reach them over an SSH tunnel:

```
# on the VM
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus
9090:9090 &
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80 &
```

```
# from your laptop
ssh -i k8s-key -L 9090:localhost:9090 -L 3000:localhost:3000 <admin>@<host>
```

Then browse `http://localhost:9090` and `http://localhost:3000`.

In Prometheus, **Status** → **Targets**: every job `UP`, nothing permanently down. Prove you have all five layers by running one query against each:

Layer	Source	Try a query against
Host	node-exporter	memory or CPU on the node
Cluster objects	kube-state-metrics	Deployment replicas in `shop`
Containers	kubelet / cAdvisor	per-container memory in `shop`
Ingress edge	ingress-nginx	controller request count
Application	the services' own `/metrics`	HTTP requests in `shop`, **and** your business counters

The fifth layer has two halves and the brief asks for both: the HTTP metrics the exporter gives you for free, and at least two business metrics that no generic exporter could produce. At least one of those two must be a counter **you** added to the application code — the supplied `orders` service already has some, and reusing only those does not meet the requirement.

> **If the application layer returns nothing** while everything else works, your ServiceMonitor has been created and silently ignored. Nothing is logged when this happens. Re-read the note at the top of `monitoring/servicemonitors.yaml` and the corresponding one in `terraform/platform/main.tf` — the two together are what make discovery work, and you should know which of them is doing the job.

Evidence for Task 6: the Targets page and the six queries with results.

---

10. Build the dashboard

Write first: nothing — build it in the Grafana UI, then export it to `monitoring/grafana-dashboard.json` and commit it. See `monitoring/README-dashboard.md` for what it must cover and how it is marked.

Import your export into a fresh Grafana before you submit. Exports often carry a datasource UID that only exists on the machine they came from, and the panels come up empty for anyone else.

Evidence for Task 7: the dashboard under load, and the committed JSON.

---

11. Generate load and raise an alert on purpose

In one terminal:

```
./scripts/generate-load.sh <host> 600 mixed 8
```

In another, watch the dashboard.

Then force an alert. Which load you use matters:

```
./scripts/generate-load.sh <host> 600 errors 8     # 4xx only
```

`errors` mode asks for a product that does not exist. That is a **client** error, so an error-rate alert filtered on 5xx — as yours should be — will correctly *not* fire. Do not conclude your rule is broken.

To produce genuine server errors, break a dependency instead:

```
kubectl -n shop scale deploy/catalog --replicas=0
./scripts/generate-load.sh <host> 600 order 8
# ... then put it back
kubectl -n shop scale deploy/catalog --replicas=2
```

The order service now returns 503 because it cannot reach the catalog. Notice *where* that 5xx appears: on the orders job, not at the ingress, because the frontend still renders a page. That is the argument for instrumenting every service rather than only the edge, and it belongs in Task 8.

Your rules should move **Inactive** → **Pending** → **Firing** once the `for` duration elapses. Capture all three states. The Pending screenshot is what proves your `for` duration exists and is doing something; without it a marker cannot tell whether you set one at all. Check Alertmanager too.

Evidence for Task 6: an alert in Pending and then Firing, with an account of how you caused it.

---

12. Run the failure drill

This produces most of the data for Task 8.

```
./scripts/failure-drill.sh <host>
```

Four experiments, with pauses so you can capture the dashboard at each stage:

	What it does	What to watch for
A	Catalog scaled to 0	Does the page still load? What exactly stops working?
B	Redis scaled to 0	Does the page still load, and is it still *fast*? Is the message different from A?
C	Catalog scaled to 5	Which Pods are new? How does traffic distribute?
D	Redis Pod deleted	What survives, and why?

Set `PAUSE=180` if you want longer between stages.

Do not just record that the system "handled it". Task 8 asks you to distinguish graceful degradation from outage and to name the design decisions that produced each behaviour. Several of those decisions are ones you made in Task 4 — the probe that does not check its dependencies, the volume that outlives the Pod — so this is where you find out whether you made them for the right reasons.

Evidence for Task 8: screenshots at each stage, plus the metric values.

---

13. Capture the terminal evidence

```
./scripts/evidence.sh <host>
ls evidence/
```

That writes about twenty files covering infrastructure, platform, images, workloads, TLS and monitoring. Screenshots you still take yourself.

---

14. Write it up

- **Implementation report** — your workflow, each component's role, and the screenshots listed under *Evidence Required* in each task of the brief.
- **Critical evaluation** — Task 8, 1500 words maximum. Use `docs/CRITICAL-EVALUATION-TEMPLATE.md`; it gives the structure and the queries that produce each number.

Twenty of the hundred marks are for Tasks 8 and 9, neither of which is about building anything. Write your reasoning down as you go. You will not reconstruct it three weeks later from a directory of YAML.

---

15. Clean up

Kubernetes first, then the cloud. Deleting Pods frees nothing at the provider.

```
helm uninstall monitoring -n monitoring
kubectl delete -f monitoring/ -f manifests/

cd terraform/platform && terraform destroy
cd ../infra-<cloud>    && terraform destroy   # the same -var flags you applied
with
terraform state list   # must be empty
```

Confirm at the provider:

```
aws ec2 describe-instances --query 'Reservations[].Instances[].State.Name'
az resource list --resource-group <rg> -o table
openstack server list
```

> To pause rather than destroy: `aws ec2 stop-instances`, `az vm deallocate` or `openstack server stop`. Compute charges stop and disks persist. On Azure anything on `/mnt` is lost when you do this; `/datadisk` survives on all three, which is why the bootstrap never uses `/mnt`.

Destroy before the due date only once you have all your evidence. You cannot re-take a screenshot of a VM that no longer exists.

Submission checklist

```
./scripts/check-progress.sh <cloud>     # nothing left TO WRITE
./scripts/verify.sh <host>              # no FAIL lines
```

- [ ] Terraform for both stages, excluding `.terraform/`, state files and keys
- [ ] Only the `infra-<cloud>` directory you actually used
- [ ] Both Dockerfiles, and the application source
- [ ] All eight manifests, with `__SHOP_HOST__` still a placeholder, not your hostname hardcoded
- [ ] The teaching example deleted from the cluster and absent from your manifests: `kubectl -n shop get ingress` lists exactly one, yours
- [ ] `monitoring/alerts.yaml` and the exported `grafana-dashboard.json`
- [ ] Implementation report with the per-task evidence
- [ ] Critical evaluation, 1500 words maximum
- [ ] `verify.sh` output
- [ ] **No secrets.** No cloud credentials, private keys, kubeconfig, state files or real passwords:

```
grep -rIn "password\|BEGIN.*PRIVATE KEY\|client_secret\|access_key" . \
  --exclude-dir=.git --exclude-dir=.terraform
```

A live credential anywhere in the archive caps Criterion 4 at *Not achieved*, however good the rest of the submission is.